AN INTEGRATED ACADEMIC PLANNING AND COURSE RATING WEB PLATFORM FOR EGCI STUDENTS

MISS TRINNAYA DAMRONGPATHARAWAT
MR. KYAW ZIN THANT
MR.NATHAN TANABORIBOON

A PROJECT REPORT SUBMITTED IN PARTIAL FULFILLMENT OF THE REQUIREMENT FOR THE DEGREE BACHELOR OF ENGINEERING IN COMPUTER ENGINEERING

FACULTY OF ENGINEERING & INTERNATIONAL COLLEGE
MAHIDOL UNIVERSITY

2025

AN INTEGRATED ACADEMIC PLANNING AND COURSE RATING WEB PLATFORM FOR EGCI STUDENTS

MISS TRINNAYA DAMRONGPATHARAWAT
MR. KYAW ZIN THANT
MR.NATHAN TANABORIBOON

A PROJECT REPORT SUBMITTED IN PARTIAL
FULFILLMENT OF THE REQUIREMENT FOR THE DEGREE
BACHELOR OF ENGINEERING IN COMPUTER ENGINEERING

FACULTY OF ENGINEERING & INTERNATIONAL COLLEGE
MAHIDOL UNIVERSITY
2025

Computer Engineering Project
entitled

AN INTEGRATED ACADEMIC PLANNING AND COURSE RATING WEB PLATFORM FOR EGCI STUDENTS

_____
MISS TRINNAYA DAMRONG-PATHARAWAT
Researcher

_____
MR. KYAW ZIN THANT
Researcher

_____
MR.NATHAN TANABORIBOON
Researcher

_____
Dr. LALITA NARUPIYAKUL, Ph.D. (Computer Science),
Ph.D. (Computer and Electrical Engineering)
Advisor

Thesis
entitled

AN INTEGRATED ACADEMIC PLANNING AND COURSE RATING WEB PLATFORM FOR EGCI STUDENTS

was submitted to the Faculty of Engineering & International College,
Mahidiol University
for the degree of Bachelor of Engineering (Computer Engineering)
on
July 15, 2024

_____
Dr. LALITA NARUPIYAKUL, Ph.D. (Computer Science),
Ph.D. (Computer and Electrical Engineering)
Chair Committee

_____
Committee 1, Ph.D. (Computer Engineering)
Committee

_____
Committee 2, Ph.D. (Computer Engineering)
Comittee

BIOGRAPHY

NAME	MISS TRINNAYA DAMRONGPATHARAWAT
DATE OF BIRTH	26 December 2004
BIRTHPLACE	Nakhon Ratchasima, Thailand
EDUCATION BACKGROUND	Mahidol International College
MOBILE PHONE	098 454 9415
E-MAIL	trinnaya.dam@student.mahidol.edu

NAME	MR. KYAW ZIN THANT
DATE OF BIRTH	11 August 2003
BIRTHPLACE	Yangon, Myanmar
EDUCATION BACKGROUND	Mahidol International College
MOBILE PHONE	082 582 5476
E-MAIL	kyawzin.tha@student.mahidol.edu

BIOGRAPHY

NAME	MR.NATHAN TANABORIBOON
DATE OF BIRTH	17 September 2006
BIRTHPLACE	Bangkok, Thailand
EDUCATION BACKGROUND	Mahidol International College
MOBILE PHONE	093 020 7448
E-MAIL	nathan.tan@student.mahidol.edu

ACKNOWLEDGEMENT

First and foremost, we would like to express our special thanks and sincere of gratitude to our advisor and co-advisors, Asst. Prof. Lalita Narupiyakul, Asst. Prof. YYY YYYY, Asst. Prof. ZZZ ZZZZ, who insight, advise and knowledge us a lot in finalizing this project within the limited time frame.

Beside our advisors, we would like to thank to the committees, Asst. Prof. XXX XXXX, Asst. Prof. YYY YYYY, and Asst. Prof. ZZZ ZZZZ, for their insightful comment and encouragement. Also, the questions which encourage us to rethink and research more about some factors which we missed. Therefore, this project would not be completed without comments, questions, and support from our committees.

Finally, we would like to special thanks to our university, Mahidol University International College, and Faculty of Engineering, Mahidol University, which provided us a lot of resources to study, financial means for researching this project.

MISS TRINNAYA DAMRONGPATHARAWAT
MR. KYAW ZIN THANT
MR.NATHAN TANABORIBOON

ชื่อโครงงาน	ชื่อวิทยานิพนธ์
ผู้วิจัย	นายชื่อผู้แต่ง นามสกุลผู้แต่ง ICCI
	นางสาว ชื่อผู้แต่ง นามสกุลผู้แต่ง ICCI
	นายชื่อผู้แต่ง นามสกุลผู้แต่ง ICCI
	ปริญญา วิศวกรรมศาสตรบัณฑิต สาขาวิชาวิศวกรรมคอมพิวเตอร์
อาจารย์ที่ปรึกษา	ดร. ชื่อที่ปรึกษา นามสกุลที่ปรึกษา
สำเร็จการศึกษา	15 กรกฎาคม 2569

บทคัดย่อ

บทคัดย่อภาษาไทย (not required for your project proposal, but it is mandatory for the black book)

คำสำคัญ : ลาเท็ก / วิทยานิพนธ์ (~5 คำ)

32 หน้า

AN INTEGRATED ACADEMIC PLANNING AND COURSE RATING WEB PLATFORM FOR EGCI STUDENTS

STUDENTS	MISS TRINNAYA DAMRONGPATHARAWAT ICCI
	MR. KYAW ZIN THANT ICCI
	MR. NATHAN TANABORIBOON ICCI
DEGREE	Bachelor of Engineering (Computer Engineering)
PROJECT ADVISOR	Dr. LALITA NARUPIYAKUL, Ph.D. (Computer Science),
	Ph.D. (Computer and Electrical Engineering)
DATE OF GRADUATION	July 15, 2024

ABSTRACT

AN INTEGRATED ACADEMIC PLANNING AND COURSE RATING WEB PLATFORM FOR EGCI STUDENTS

An abstract (250-400 words) should provide a concise summary of your entire thesis. It should report significant elements of your thesis including background or introduction in brief, objectives, statistical data, key findings, and conclusion. Mathematical formulas, diagrams, and other illustrativematerials are not recommended for inclusion. A strong abstract should be self-contained; without abbreviations, footnotes, references. Outside readers typically view the abstract before deciding to read the thesis, so it should be well written, logical, and a complete reflection on you work. The abstract is typically written last after finishing chapter 4 and 5.

KEYWORDS : LaTeX / Thesis (~5 words)

32 Pages

CONTENTS

	Page
BIOGRAPHY	**i**
ACKNOWLEDGEMENT	**iii**
ABSTRACT (THAI)	**iv**
ABSTRACT (ENGLISH)	**v**
CONTENTS	**vi**
LIST OF TABLES	**ix**
LIST OF FIGURES	**x**
CHAPTER 1 INTRODUCTION	**1**
1.1 Background	1
1.2 Objectives	2
1.3 Scope	3
1.4 Data restriction for each component	3
1.5 Expected Results	4
1.6 Timeline	5
CHAPTER 2 LITERATURE REVIEW	**6**
2.1 technological stack and system architecture	6
2.1.1 Design web service academic information system based multi-platform [9]	6
2.1.2 Design and Implementation of a Digital Notice Board and Forum Discussion System [11]	7
2.1.3 Course Selling Website Using MERN Stack [12]	7
2.2 Functions	8
2.2.1 A hyper-heuristic algorithm based on genetic and greedy strategy for university course scheduling problem [13]	8
2.2.2 A Review of Strategies for Designing, Administering, and Using Student Ratings of Instruction [7]	9

2.2.3 CourseViewer a prototype for visualizing undergraduate courses and student transcripts [3] 9
2.2.4 The curriculum prerequisite network: a tool for visualizing and analyzing academic curricula [2] 10
2.3 Contribution 11
CHAPTER 3 METHODOLOGY 12
3.1 System Architecture 12
3.1.1 Presentation Layer (Frontend) 12
3.1.2 Application Layer (Backend) 13
3.1.3 Data Management 15
3.1.4 Overall Data Flow 15
3.2 Schedule Matcher Component 15
3.2.1 Description 15
3.2.2 Processing Pipeline 17
3.2.3 Frontend 18
3.2.4 Backend, API, and Database 18
3.3 Rate-My-Course Component 19
3.3.1 Description 19
3.3.2 Processing Pipeline 21
3.3.3 Frontend 22
3.3.4 Backend and API 23
3.3.5 Database Schema 23
3.4 Super Planner Component 24
3.4.1 Description 24
3.4.2 Processing Pipeline 26
3.4.3 Progress Markers 27
3.4.4 Frontend 27
3.4.5 Backend, API, and Database 28
CHAPTER 4 RESULT 29
4.1 Results 29
4.2 Discussions 29

CHAPTER 5 Conclusion **30**

5.1 Conclusion 30

5.2 Obstacles 30

5.3 Future Work 30

REFERENCES **31**

LIST OF TABLES

Table **Page**

1.1 Project Timeline 5

LIST OF FIGURES

Figure		Page
3.1	System Architecture	12
3.2	System Authentication Flow	14
3.3	Schedule Matcher Activity Diagram	16
3.4	Schedule Matcher Pipeline	18
3.5	Rate-My-Courses Activity Diagram	20
3.6	Rate-My-Course Pipeline	22
3.7	Super Planner Activity Diagram	25
3.8	Super Planner Pipeline	27

CHAPTER 1
INTRODUCTION

1.1 Background

Finding an available time slot between friends is also a time-consuming struggle as students need to look up each other's schedule constantly. Academic planning is especially important for engineering students who must follow a structured curriculum with multiple prerequisites and limited courses offered each semester. Evaluating course difficulty, reviewing the course curriculum and coordinating schedules with peers for group study are also the struggles that most students are facing in the faculty. At the EGCI program, students currently lack a centralized platform that allows them to find available time through class-schedule screenshot, plan their academic paths in a visual and structured way, and rate the course.

Comparing schedules with friends to find common free time requires manual checking which is time consuming and error prone when there are multiple students. The process of just using the screenshots of the schedule and finding the available times would be a great solution for meetings and collaborations. In addition, the issues of the manual schedule coordinations of students can also be solved by the combination of TableExtractNet, by Ngubane and Tapamo [8] a deep learning approach utilizing CornerNet and Faster R-CNN to detect and recognize table structures from document images, which can be implemented and trained to turn the photos/screenshots of class schedules into a digital form, utilizing the "Master Busy Map" logic [4] to further develop an algorithm to discover the common available of many class schedules.

Course feedback is only shared when students ask questions in LINE group chats and there is no platform where information can be accessed any time. Several systems such as online course platforms and academic planning tools have been developed to address individual aspects of this problem. For example, "RateMyProfessors" website [1], developed to allow students to share their opinions and comments of their professors with colleagues.

Students also face difficulties when planning their academic paths. Most students rely on the student handbook to check course requirements, prerequisites, and credit conditions. However, the handbook only serves as a reference document and does not provide interactive guidance. As a result, many students become confused about which general education courses they still need to take, which electives are required, and how their course selections affect their graduation timeline. It is common for students to enroll in available courses without a clear long-term plan, which may lead to delayed graduation or inefficient course sequencing. Stellic and DegreesWork [5] [6], academic planning websites, designed to let students stay informed of their academic performance and possible academic career path that they can choose. However, most of these systems focus on single functions and lack certain capability to solve our addressed issues, for example, [1] lack a lot of information about EGCI curriculum structures, Stellic and DegreesWork [5] [6] use text-based academic planning, which [10] addressed that graph-based representations of course prerequisites can improve student's understanding of course syllabus.

The objective of this project is to develop EGCEye, an integrated academic planning and course rating web platform specifically for EGCI students. The system will include three main functions: a class schedule matcher using image processing techniques, a rate-my-course feature for student reviews and feedback, and a super visualized planner that guides students through their academic progress toward graduation.

1.2 Objectives

- To analyze user behaviours regarding academic tool usage among EGCI students, including how students currently share and compare class schedules with peers, what course-related information students seek before enrollment such as course difficulty, workload, and professor teaching style, and how students approach long-term academic planning in terms of tracking prerequisites, selecting elective courses, and managing their progression toward graduation
- To design a web interface and system architecture that integrates:
    a. A Schedule Matcher interface for image uploads
    b. A Rate-My-Course database scheme for multi-section feedback

c. A Super Planner visualization engine for prerequisite tracking on an interactive drag-and-drop interface.

- To develop a website for EGCI students to:
  a. Share their class schedules with friends to find common available times
  b. Rate courses and leave comments on the courses offered by MUIC and the EGCI faculty
  c. Create an interactive academic plan to track both courses they intend to enroll in and those they have already completed throughout their studies in the EGCI faculty
- To evaluate the website's performance in terms of image extraction accuracy, user engagement across all functions, and the overall user experience (UX).

1.3 Scope

- Platform Availability: The system will be developed as a web application, available specifically for EGCI students.
- Technology Stack
  a. Full-Stack Framework: MERN, MongoDB for database, Express.js as a backend framework, React as a frontend library, and Node.js a runtime environment to run JavaScript code
  b. Image Processing Engine: A JavaScript-based service utilizing Tesseract.js for optical character recognition and OpenCV.js for color segmentation and coordinate mapping in schedule extraction, integrated directly within the Node.js backend
  c. Programming Languages: JavaScript (ES6+) for both web architecture and image data processing

1.4 Data restriction for each component

- Schedule matcher
  a. The system accepts image uploads (JPEG or PNG) of class schedules exported from Sky+ application.

b. Date and time labels are extracted from the schedule grid by the Optical Character Recognition via Tesseract.js.
c. Extracted schedule data is stored temporarily for the duration of the matching session, and is not retained after that.

- Rate-My-Course
  a. List of courses available to review covers Major Required Courses, Engineering Core Courses, and Computer Engineering Elective Courses offered in MUIC catalog 2022.
  b. EGCI students are allowed to provide 0 to 5 star ratings, and leave a comment whether anonymously or not.
  c. Any available courses will also be separated by professors in case the course has more than one section, which are taught by different professors.
  d. Users may submit courses additional to the initial setting.
- Super planner
  a. List of courses available for drag-and-drop includes MUIC General Education courses, Computer Engineering Major Required Courses, Engineering Core Courses, and Computer Engineering Elective Courses in the catalog.
  b. This acts as a tool for visualization, not connected to MUIC official course enrollment.

1.5 Expected Results

1. A set of requirements as a foundation for the system
2. A user-centric interface design
3. Web Platform, the primary deliverable integrated with three core functions
4. Performance Evaluation based on following metrics:
   - The proposed system is expected to achieve a user base of 12 or more.
   - The Rate-My-Course reaches the number for activities: 5 for adding star-rating and tags, and 2 for leaving comments.
   - The Class Schedule Matcher targets a number of 10 screenshots uploaded

to find common free times, with all being accurately matched.

- The Super Planner aims to facilitate at least 3 unique student plans.
- The project aims to collect User Experience (UX) based on quality feedback collected from a random group of users.

1.6 Timeline

Table 1.1 Project Timeline

Plan	Timeline 2026										
	Feb	Mar	Apr	May	Jun	Jul	Aug	Sep	Oct	Nov	Dec
1: Literature Review											
2: System Arch. Design											
3: UI/UX Deign											
4: Rate My Course											
5: Schedule Matcher											
6: Super Planner											
7: Testing											
8: Documentation											
9: Final Presentation											

CHAPTER 2
LITERATURE REVIEW

This chapter discusses the published academic papers used to support our project. There are two main parts: The first part discussed the technology stack, website development framework, and website development planning that has been implemented and met with success. Such technology stack and planning would then be used as a guideline to our project development. The second part explores any methods or algorithms that could be adapted and used for the project's main functions development. For this part, it will be divided into 3 sub-parts for 3 functions, with each part containing reviewed literature that are related to the function of that part.

2.1 technological stack and system architecture

2.1.1 Design web service academic information system based multiplatform [9]

This paper proposed a system design that uses Web Services to make academic information available on both web and mobile platforms.

The core of the design is a REST API that uses JSON to exchange data between the server and different types of devices. This approach was chosen because it allows a single database to support many different user interfaces while keeping all the information synchronized in real-time.

The study found that using this web service architecture makes the system much easier to maintain and scale up because the "business logic" on the server is kept separate from how the app looks on the screen.

For future work, the authors suggested adding stronger security protocols to the API and making the database faster so it can handle more students using the app at the same time.

2.1.2 Design and Implementation of a Digital Notice Board and Forum Discussion System [11]

This paper presents an educational website, equipped with two functions, digital notice board and discussion forum, which are developed to fulfill two goals that aim to improve university students' quality of life. The first goal is to create a centralized space for information dissemination, to ensure that any relevant parties can stay informed of any news or announcement. Second goal is to create an easy-to-access educational digital discussion platform in order to foster communication and discussions among students university staff.

Technically, the website was built using MERN stack (MongoDB, Express.js, React.js, and Node.js), which is a website development framework that is famous for its quick development time, ease of use, and its compatibility with JavaScript language, with the framework being complete on its own, covering front end, back end, API, and database, all within one framework.

The research concludes that transitioning to a digital framework significantly improves user engagement and reduces the risk of miscommunication. For future development, the author suggests further optimizing the user experience and exploring additional features.

2.1.3 Course Selling Website Using MERN Stack [12]

This paper presents an educational and commercial website that lets university teacher to create, manage, and sell their courses online, intended to be a platform that improves the education quality by narrowing the gap between student's demand for educational courses and teacher's courses supply under the platform equipped with firm security and secure user's privacy protection.

With the website development heavily implements the MERN stack (MongoDB, Express.js, React.js, Node.js) which is a website development framework that is known for its quick development time, ease of use, and its high compatibility with JavaScript language. Additionally, the website also implements the REST API to obtain website's fast response time, and the capability to handle a large number of users and website's course data without being slowed down.

The study found that this architecture makes the platform very flexible, allowing for features like real-time chat and secure payments to be added easily. By keeping the "business side" of the website separate from the "user side," the system is easier to update as more students join.

For future work, the author suggests adding mobile options for the website, AI assistance to help answer student's questions, and digital certificate to serve as a proof when students finish their courses.

2.2 Functions

Schedule matcher

2.2.1 A hyper-heuristic algorithm based on genetic and greedy strategy for university course scheduling problem [13]

This study addresses the University Course Scheduling Problem (UCSP) by proposing a novel model that balances resources like classrooms and teachers while satisfying complex constraints.

A hyper-heuristic algorithm (GA_RG_HH) was developed, combining Genetic Algorithms (GA) with greedy strategies to optimize scheduling efficiency.

Performance was evaluated using real-world datasets, measuring the reduction of soft constraint violations and the efficiency of resource allocation.

The experiment indicated that the proposed GA_RG_HH performs better than traditional algorithms like Tabu Search and Particle Swarm Optimization. This is attributed to the algorithm's ability to balance exploration and exploitation through a pre-check strategy and dynamic operator sequences.

Future work includes further optimization of the algorithm for even larger-scale institutional data and the inclusion of more diverse constraint parameters.

Course rater

2.2.2 A Review of Strategies for Designing, Administering, and Using Student Ratings of Instruction [7]

This paper reviews the best ways to design and use student feedback (course evaluations) to help teachers improve their classes, specifically in medical and pharmacy programs.

The core of the study suggests that a good evaluation should be carefully planned to be fair and useful. The author recommended using a standardized online forms with about 10 to 30 questions that are easy to understand for students. The reason why the forms are suggested to be in online format is because it allows the results to be collected and processed more efficiently compared to the traditional paper format. The research also recommended that there should be a formal scheduled time for students to complete the survey, for example, during class period, to prevent "survey fatigue".

Additionally, the research found that the evaluations are most effective when they focus on the "teaching aspect" of the courses, for example, the goal of the course, and how well the the teachers prepare and present their materials. The study also pointed out that although online evaluations receive less response than paper evaluations, the quality of the results is usually the same.

For future work, the author suggests using random samples with the survey, since the traditional method of collecting course evaluations is to distribute the survey to every students, but by using random sampling (selecting a group of students in random), it can reduce the time taken to collect the evaluations while maintaining the quality of the results. The teacher's access to the evaluation data is also advised to be limited for better efficiency in data distribution.

Super planner

2.2.3 CourseViewer a prototype for visualizing undergraduate courses and student transcripts [3]

This paper introduces CourseViewer, a prototype that transforms traditional, text-heavy transcripts into interactive visual maps to help students plan their academic paths. This approach is highly relevant for an academic planner because it

replaces manual checking with a dynamic system that shows how different courses are connected.

The most useful feature for a modern planner is the graph-based visualization. By representing courses as "nodes" and prerequisites as directed arrows, the system makes it easy to see which classes "unlock" future subjects. This technical design helps prevent enrollment errors by clearly showing if a student has met the necessary requirements before they try to add a class.

Technically, the tool uses the Java language and the Prefuse visualization toolkit to automatically handle the map layout so the user doesn't have to organize it manually. It also uses color-coding—such as green for completed and red for required—which provides an immediate visual status of a student's progress toward graduation.

The study found that these visual maps allow students to identify their remaining credits much faster and more accurately than reading a paper transcript. For future work, the authors suggested adding a "what-if" feature to simulate different majors and integrating the tool directly into online registration systems.

2.2.4 The curriculum prerequisite network: a tool for visualizing and analyzing academic curricula [2]

This paper treats a university curriculum as a "complex system" that can be mapped using network science to reveal the hidden connections between courses. This is directly applicable to the project's goal of automating 4-year academic planning for major and general education courses, which currently must be done manually.

The study uses a Directed Acyclic Graph (DAG) where courses are "nodes" and prerequisites are "links". A key technical detail is the identification of "hubs" (courses that unlock many others) and "bridges" (links between different course groups). Implementing this logic allows the interactive planner to visually flag "bottleneck" courses, helping students avoid choices that could delay their graduation by a year.

Technically, the research utilized NetworkX and Gephi to analyze network density and "betweenness centrality". In the project, these metrics can be used to calculate the "long-term impact" of taking a specific class, providing an automated

"roadmap" that ensures all prerequisite constraints are met at a glance.

2.3 Contribution

The literature reviewed in this chapter provides both technical and conceptual foundations that directly inform the design and development of EGCEye.

For the technology stack, the MERN-based implementations demonstrated in [11] and [12] confirm that MongoDB, Express.js, React, and Node.js form a proven and cohesive stack for educational web platforms at a similar scope to EGCEye. The REST API design principles outlined in [9] further reinforce that a single API layer can serve multiple interface types while maintaining data consistency — an architecture that EGCEye directly adopts.

For the Schedule Matcher, the Master Busy Map logic [4] provides the core algorithm for deriving common free windows across multiple students' schedules, while TableExtractNet [8] informs the image-based table extraction approach used to convert schedule screenshots into structured data.

For the Rate-My-Course function, [7] provides evidence-based guidance on feedback system design, supporting EGCEye's decision to keep star rating as the only required field, list courses separately by professor where multiple sections exist, and present aggregated results in a structured and accessible format.

For the Super Planner, CourseViewer [3] serves as the direct precedent for the node-and-arrow graph representation and the color-coding of course completion states. The curriculum prerequisite network study [2] further supports the use of graph-based visualization over text-based planning tools, and informs the planner's bottleneck warning system that flags courses a student should prioritize based on their year level.

CHAPTER 3
METHODOLOGY

3.1 System Architecture

The system is designed as a web-based platform built on a multi-layered architecture that separates into three distinct layers: the Presentation Layer, the Application Layer, and the Data Management Layer. This separation ensures modularity,maintainability, and a clear division of responsibilities across the system.

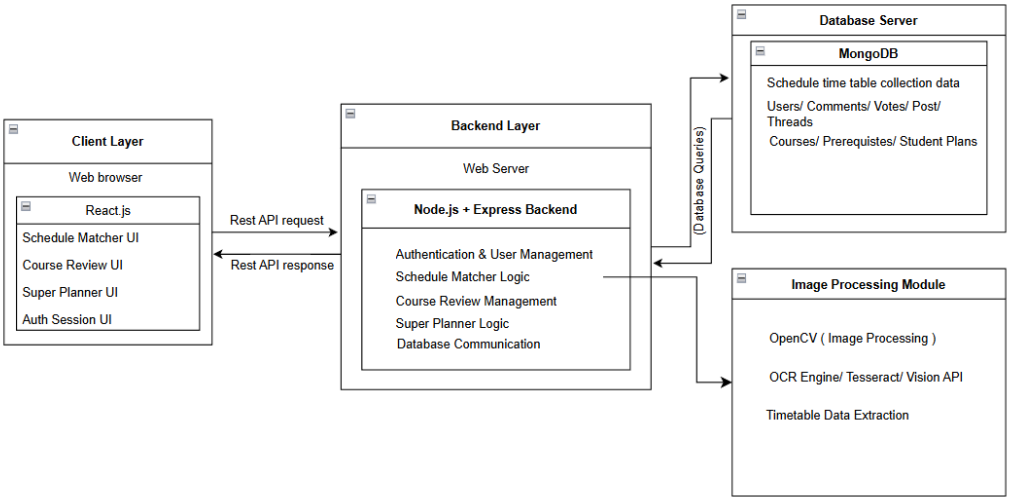

Figure 3.1 System Architecture

3.1.1 Presentation Layer (Frontend)

The Frontend of the system is built using React, a component-based JavaScript library that enables dynamic and responsive user interfaces. Each of the three core functions — Schedule Matcher, Rate-My-Course, and Super Planner — is implemented as an independent module where users can navigate between features.

The interface is designed with a focus on simplicity and visual clarity, which is especially important for the Super Planner's drag-and-drop canvas and the Schedule Matcher's result display. React state management handles real-time UI up-

dates, such as rendering matched free-time slots after scheduled images are processed, or updating course ratings as new submissions come in.

The Authentication Session UI is handled at the presentation layer, where users see and interact with related to their login state, for example: the login page, displaying the logged-in student's name or ID on the taskbar, a logout button, disabling or hiding buttons based on whether the user is logged in, and protected routes redirected.

3.1.2 Application Layer (Backend)

The Backend is built using Node.js and Express.js, forming a RESTful API server that handles all communication between the frontend and the database. Express.js routes are organized by function, with separate route groups for schedule matching, course rating, and academic planning.

Image processing for the Schedule Matcher is handled entirely within this layer using JavaScript-native libraries — Tesseract.js for OCR and OpenCV.js for color segmentation This keeps the entire stack in a single language, simplifies deployment, and reduces points of failure.

Authentication is also enforced at this layer through Express.js middleware, which validates tokens on all protected API routes and rejects unauthorized requests. This ensures that only EGCI students can access and contribute to the platform.

- Verifying credentials (checking the submitted student ID/password against the database)
- Issuing tokens (e.g. JWT) after successful login
- Protecting API routes via middleware that checks for a valid token on every request

As shown in Figure 3.2

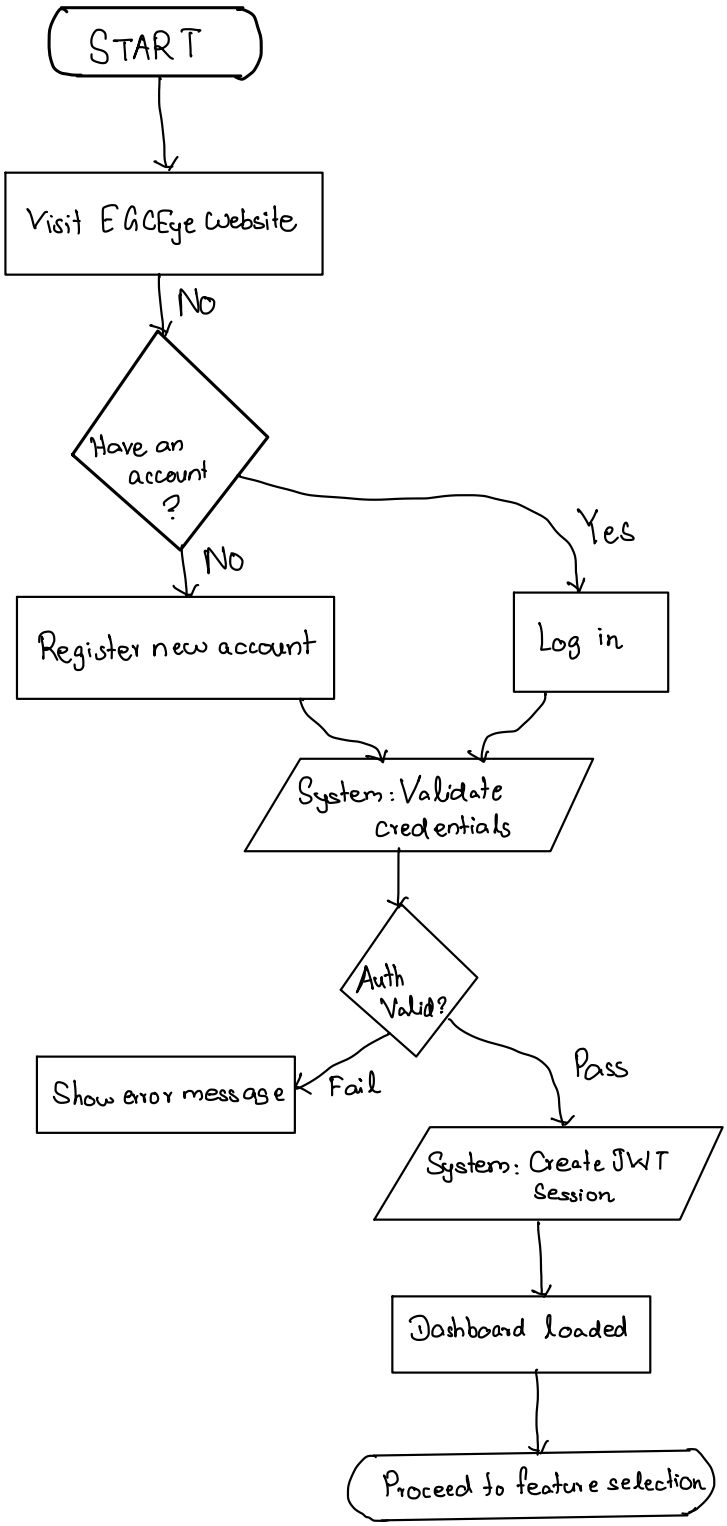

Figure 3.2 System Authentication Flow

3.1.3 Data Management

The system uses MongoDB, a NoSQL document-oriented database, as its primary data store practice. MongoDB's flexible schema design suits well with our web platform since all the three functions store structurally different data.

Course reviews contain nested rating objects and optional comment fields, while academic plans store arrays of semester slots each containing course objects with prerequisite relationships.

Collections are organized around the three core functions: a schedules collection for temporarily storing extracted schedule data during matching session, a courses collection for course metadata and aggregated ratings, a reviews collection for individual student submissions, and a plans collection for saved Super Planner layouts.

3.1.4 Overall Data Flow

When a user interacts with the React frontend, requests are sent to the Express.js API server. Depending on the request type, the backend either queries MongoDB directly — for course data, reviews, or saved plans — or triggers the image processing pipeline locally for schedule matching. Results are returned as structured JSON responses to the frontend for rendering. This fully unified architecture means the entire platform can be deployed as a single backend service alongside the React frontend, significantly reducing deployment complexity.

3.2 Schedule Matcher Component

3.2.1 Description

The Class Schedule Matcher allows EGCI students to upload screenshots of their class schedules exported from the Sky+ system and automatically identify common free time slots across a group. This removes the need for manual schedule comparison, which becomes increasingly error-prone as the number of students in a group grows. The function relies on two core image processing techniques: Optical Character Recognition (OCR) via Tesseract.js, which extracts the text-based day and time labels that define the structure of the schedule grid, and color segmentation via OpenCV.js, which detects the colored rectangular blocks representing occupied class periods. To-

gether these two techniques convert a raw schedule screenshot into structured time slot data that can then be compared across multiple students to find common availability.

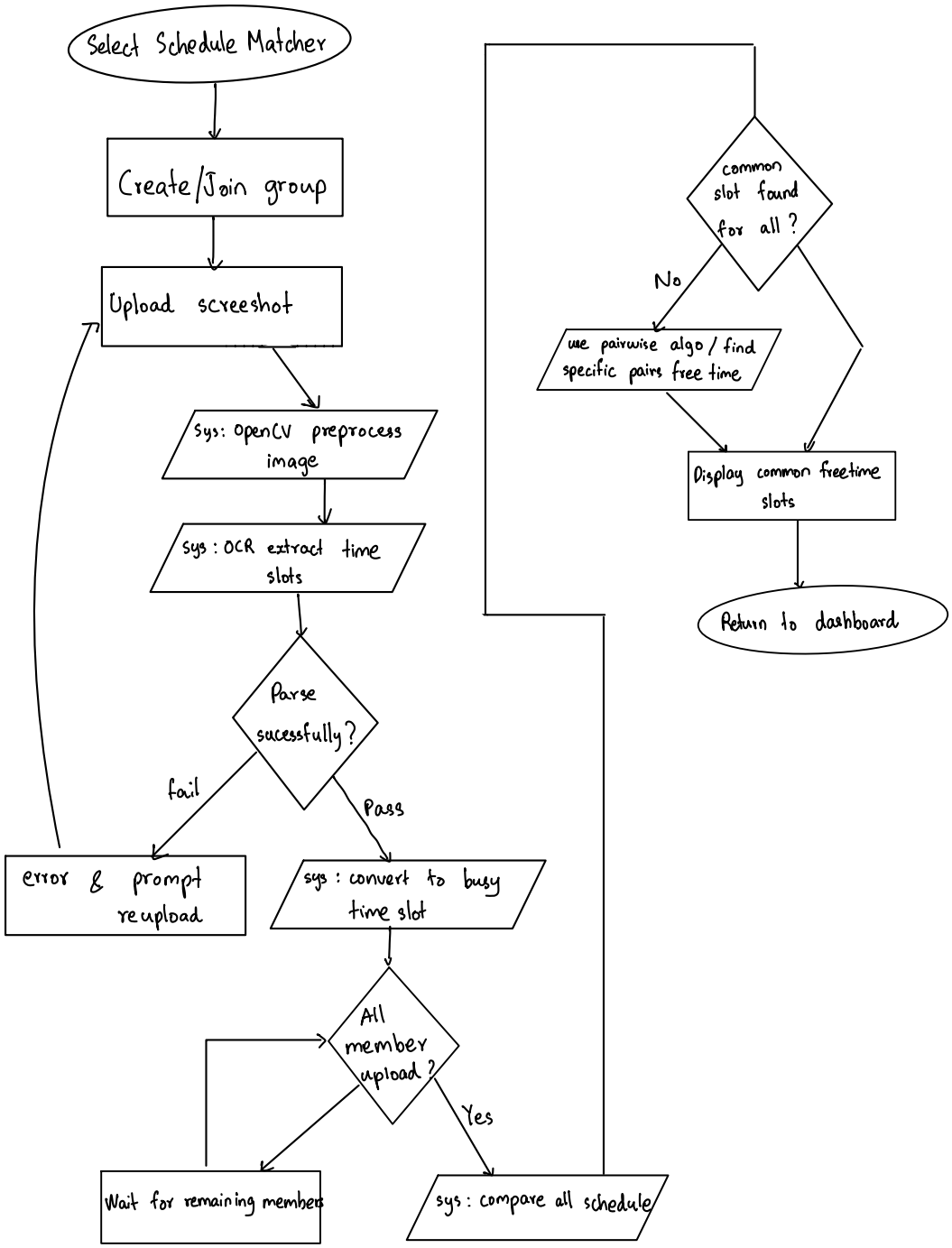

Figure 3.3 Schedule Matcher Activity Diagram

3.2.2 Processing Pipeline

Schedule image upload Based on the Sky+ schedule format, the system receives screenshots that follow a consistent and predictable structure. The schedule is presented as a grid with day labels (SUN through SAT) along the vertical axis and time markers (08:00 through 20:00) along the horizontal axis. Occupied time slots are represented as solid-colored rectangular blocks placed within the grid. The grid itself sits on a white card background, providing high contrast against the colored blocks. Block colors may vary between students and between courses, but the rectangular shape and clean edges remain consistent across all inputs.

Grid Detection and Cropping The backend first isolates the schedule grid from the rest of the screenshot. Since the Sky+ schedule card consistently appears as a white-background region with clearly defined borders, the system detects the bounding box of the grid area using edge detection in OpenCV.js and crops to that region before any further processing is done. This eliminates any interference from the background wallpaper visible around the schedule card.

Column and Row Calibration Tesseract.js is applied to the cropped image to extract the time labels along the top of the grid (e.g. 08:00, 09:30, 11:00) and the day labels along the left side (e.g. MON, TUE). These extracted labels and their pixel positions are used to build a coordinate map that translates any horizontal pixel range into a corresponding time interval and any vertical pixel range into a corresponding day.

Color Blob Detection OpenCV.js scans each row of the calibrated grid and identifies horizontal spans of pixels that are a colored course blob. Since the blocks are solid rectangles with clean edges against the white grid background, simple color thresholding is sufficient to detect them reliably without requiring more complex segmentation techniques. Each detected span is recorded as a start pixel and end pixel within a given day row.

Busy Map Construction and Matching The pixel spans from previous step are mapped to actual time slots using the coordinate map built in Step 3, producing a structured busy map for each uploaded schedule. Once all students' busy maps have been generated, the Master Busy Map algorithm performs an OR logic operation across them to identify time slots where every student is simultaneously free. If no such slot exists

for the full group, the system performs a subgroup fallback by removing one student at a time and repeating the matching process, surfacing the best available options for partial groups.

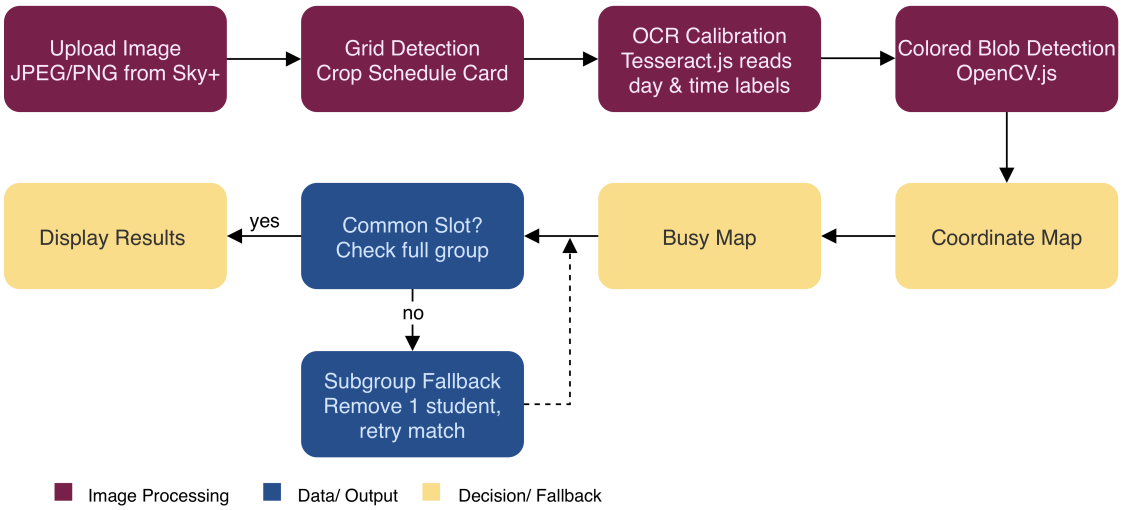

Figure 3.4 Schedule Matcher Pipeline

3.2.3 Frontend

The Schedule Matcher interface allows users to upload multiple schedule images and view the resulting common free time slots displayed in a visual timetable. Upload progress and processing status are surfaced in real time so users remain informed throughout the process.

3.2.4 Backend, API, and Database

The Express.js backend exposes an upload endpoint that receives image files from the frontend and passes them through the processing pipeline described in Section 3.2.2. Once processing is complete, the structured busy map data for each uploaded schedule is temporarily written to the schedules collection in MongoDB before the matching algorithm runs across all uploaded schedules in the session.

Each document in the schedules collection follows a lightweight structure containing a session identifier, an optional student label, a timestamp, and the extracted busy map represented as an array of occupied time slot objects. Each time slot object holds a day field and a start and end time derived from the coordinate mapping

step.

The session identifier groups all uploaded schedules belonging to the same matching request together, allowing the Master Busy Map algorithm to retrieve and compare all relevant documents in a single query. Once the matching results have been computed and returned to the frontend, all documents sharing that session identifier are deleted from the collection. This ensures that the schedules collection functions purely as a short-lived working space rather than a persistent data store, keeping the database lean and avoiding unnecessary retention of student schedule data.

3.3 Rate-My-Course Component

3.3.1 Description

The Rate-My-Course function allows EGCI students to rate the any available General Education courses, major required course, and engineering core course offered by MUIC and EGCI faculty. Students can submit a star rating from 0 to 5 and optionally leave a written comment, whether anonymously or not. The course catalog available for review covers Major Required Courses, Engineering Core Courses, and Computer Engineering Elective Courses offered in the MUIC 2022 catalog. Where a course has more than one section taught by different professors, the course is listed separately per professor, allowing students to give feedback that is specific to their actual classroom experience rather than the course in general. Students may also submit courses that are not present in the initial catalog, allowing the list to expand over time as new courses become available.

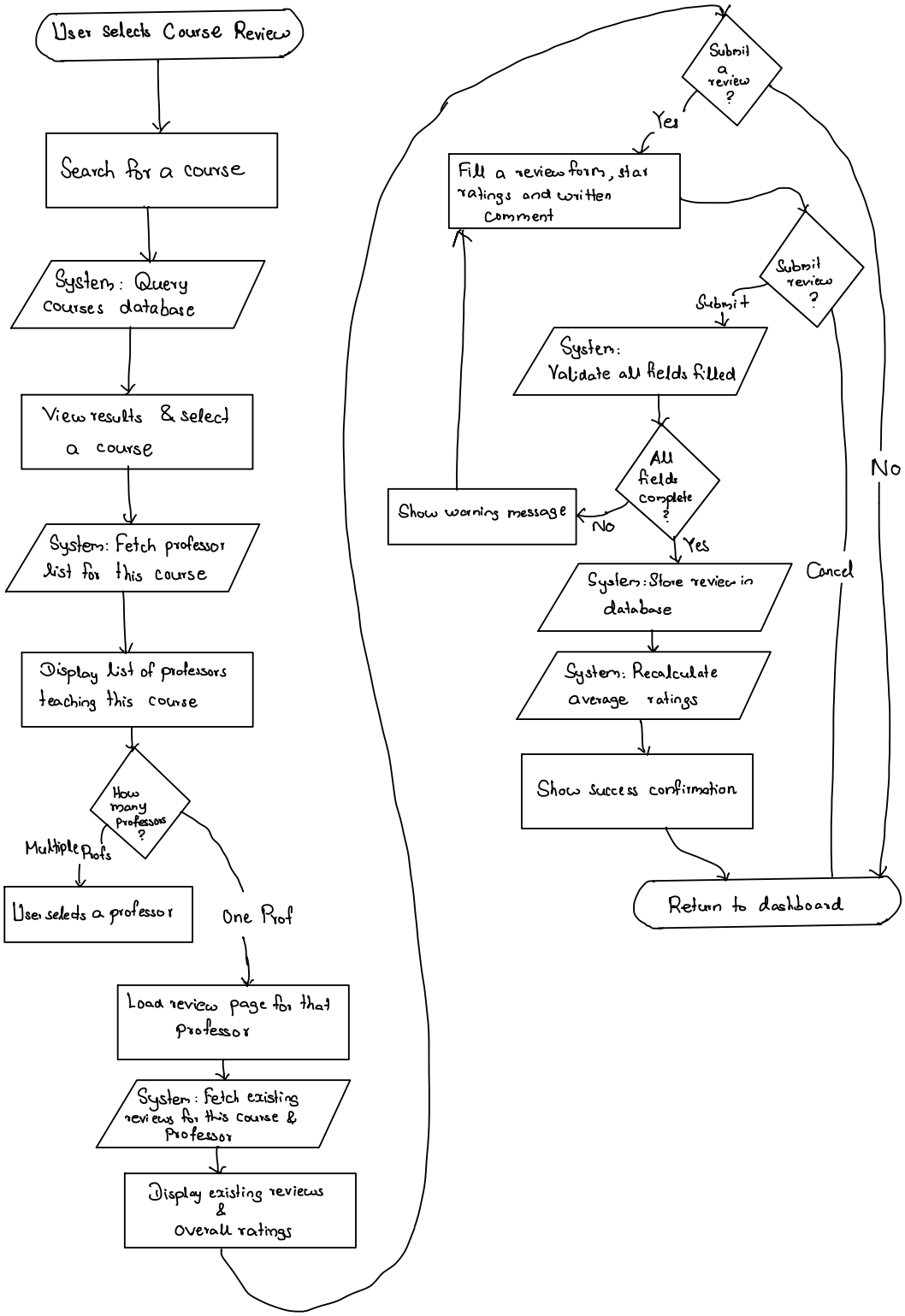

Figure 3.5 Rate-My-Courses Activity Diagram

3.3.2 Processing Pipeline

Browe and Read Pipeline: The pipeline begins when a student lands on the Rate-My-Course page, where the trending dashboard is loaded by querying the reviews collection for the most recently submitted reviews, grouped by course, and returning the top results to the frontend for display. If the student is looking for a specific course, they enter a course code or course name into the search bar, which sends a query to the backend courses endpoint. The backend searches the courses collection for matching documents and returns the results to the frontend as a list. The student selects a course from the results, and the course page is loaded by fetching the course metadata from the courses collection alongside a dynamic aggregation query that computes the current average rating and total review count from the reviews collection. Individual reviews for the selected course section are also retrieved and displayed in chronological order.

Review Submission pipeline: Once the student is on a course page, they can choose to submit a review. The submission form collects a star rating, an optional written comment, and an anonymity toggle. Basic client-side validation confirms that a star rating has been selected before the form can be submitted. The review payload — containing the course code, section number, star rating, optional comment, anonymity flag, and student ID derived from the authenticated session — is then sent to the backend via a POST request to the reviews endpoint.

On the backend, the payload undergoes server-side validation to confirm that the rating is a numeric value between 0 and 5, that the referenced course and section exist in the courses collection, and that all required fields are present and correctly typed. If validation passes, the review document is written to the reviews collection with a server-generated timestamp. If any validation check fails, an error response is returned and nothing is written to the database.

Once a review is successfully stored, any subsequent request to the course page triggers a fresh aggregation query to recompute the average rating and total review count, ensuring that the displayed statistics always reflect the most current data.

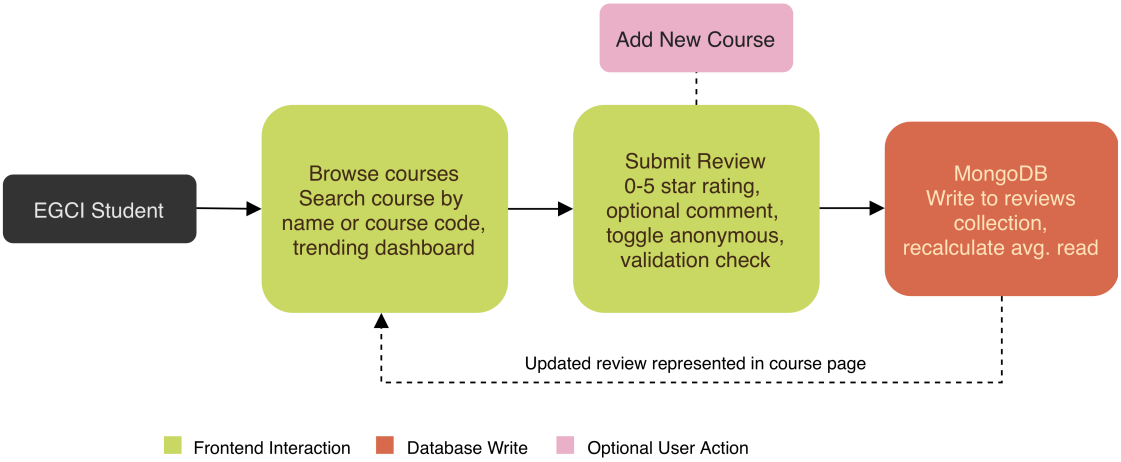

Figure 3.6 Rate-My-Course Pipeline

3.3.3 Frontend

The Rate-My-Course interface presents a searchable and filterable list of courses as its main view. Students can search by course code or course name using a search bar. Where a course has multiple sections under different professors, each section is displayed as a separate entry so that students can navigate directly to the professor they studied under.

The main page also features a trending courses dashboard, which highlights courses that have received the most recent review activity. This gives students an at-a-glance view of which courses are currently being discussed without requiring them to search for a specific course first, making the platform more discoverable and encouraging broader engagement across the catalog.

Each course page displays the aggregated star rating shown as both a numerical average and a visual star indicator, a breakdown of the rating distribution across all five star levels, and a chronological list of individual reviews. Reviews marked as anonymous display without a student identifier, while non-anonymous reviews show the submitter's name.

A submission form is accessible from the course page, allowing students to select a star rating by clicking on a star indicator, write an optional comment in a text field, and toggle whether their submission should appear anonymously before posting. Basic client-side validation ensures that a star rating is selected before the form can be

submitted, since a rating is the only required field. The form also includes a separate flow for submitting a new course that does not yet appear in the catalog, prompting the student to provide the course code, course name, category, and professor name.

3.3.4 Backend and API

The backend exposes a set of RESTful API endpoints organized two primary resources: courses and reviews. The course endpoints handle retrieving the course page of the student's search with its aggregated rating , and accepting new course submissions. The review endpoints handle submitting a new review for a specific course-professor pair and retrieving all reviews associated with that course section ordered by submission date and time.

All review submissions pass through server-side validation before being written to the database. The validation layer checks that the star rating is a numeric value between 0 and 5, that the course and section being reviewed exist in the database, and that the anonymity flag is a valid boolean. If any of these checks fail, the endpoint returns an appropriate error response without writing to the database. Aggregated ratings — the average score and total review count displayed on each course page — are computed dynamically from the reviews collection at read time rather than stored as fixed values, ensuring that displayed scores always reflect the most current data without requiring update operations on the course document itself.

3.3.5 Database Schema

The Rate-My-Course function uses two database collections: courses and reviews. The courses collection stores the metadata for each course entry. Each document contains course code, course name, category, and an array of sections objects. Each section object holds a name of the professor teaching in that section. This nested structure keeps different-professor within the same course grouped within a single document while still allowing it to be linked to a distinct section.

The reviews section stores each individual student review submission as a separate document. Each document consists of course code, professor, course star rating, optional comment section, anonymity boolean flag, the submitter's student ID

(stored internally but masked if anonymous), and a timestamp of when the review is created.

3.4 Super Planner Component

3.4.1 Description

The Super Planner is a visual academic planning tool with interactive design that allows EGCI students to to arrange the plan for their entire academic studies by dragging and dropping course bubbles into free trimester slots. It is intended as a personal aid rather than the official enrollment system.

The list of courses available covers MUIC General Education courses, EGCI Major Required Courses, Core courses, and Elective Courses from the MUIC catalog. Each course that has prerequisite courses will have a line connecting to the prerequisite courses. There would also be an additional “suggestion” line, which connects between a course and other available courses which are suggested by the faculty to study first although the course is not connected by the prerequisite line. With the general education courses, I-design courses, and major electives courses, the website will allow users to specify what course they would take to complete the required credit of that category.

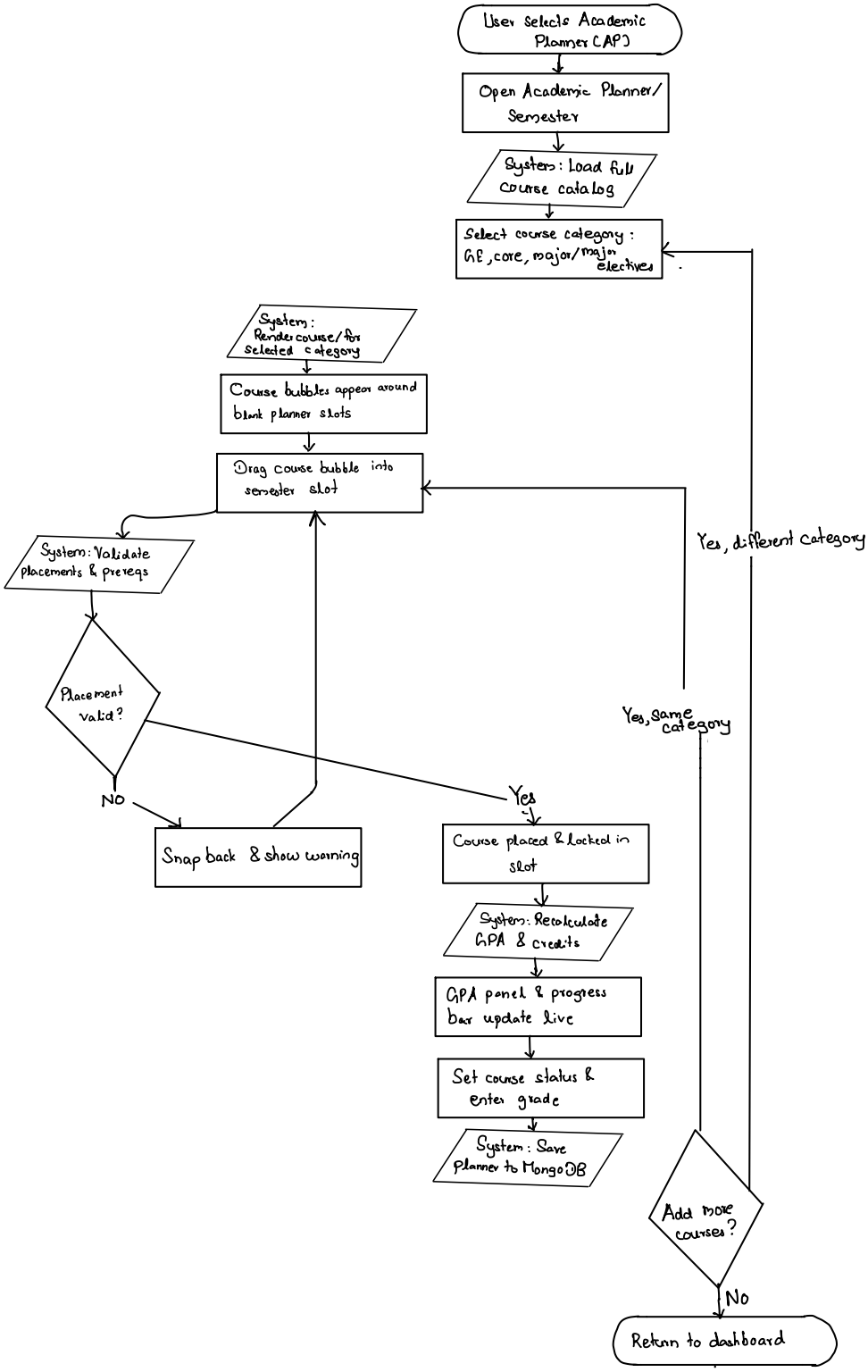

Figure 3.7 Super Planner Activity Diagram

3.4.2 Processing Pipeline

The Super Planner Pipeline covers main three flows: initializing the canvas, interacting with the plan, and auto saving.

Initialization Pipeline: When a student opens the Super Planner, the frontend simultaneously sends two requests to the backend. The first fetches the full course list from the courses collection, including each course's prerequisite relationships and faculty suggestion metadata, which is used to populate the course palette and prepare the prerequisite graph engine. The second fetches any previously saved plan from the plans collection using the authenticated student's ID.

Interaction Pipeline: When a student drags a course icon from the palette and drops it into a semester slot, the prerequisite graph engine checks whether any of that course's prerequisites or suggested courses are already placed on the canvas and immediately renders the appropriate connecting lines. If a prerequisite course has not yet been placed, the line is rendered in a warning color to indicate an unresolved dependency.

When a student moves a course from one semester slot to another, all connecting lines associated with that course are recomputed and redrawn based on the new position, and the progress marker system reruns its check to update any warning markers that may have changed as a result of the move.

When a student marks a course as completed, the course icon updates its visual state on the canvas and the progress marker check reruns to reflect the updated academic standing.

When a student removes a course from the canvas and returns it to the palette, all connecting lines to and from that course are removed, dependent courses that relied on it as a prerequisite have their lines updated to reflect the unresolved dependency, and the progress marker system reruns to reattach a warning marker to the returned course if applicable.

Save Pipeline: The Super Planner uses an autosave mechanism to persist the student's plan automatically without requiring a manual save action. After every interaction — a course placement, move, completion toggle, or removal — a debounce timer of approximately one to two seconds is reset. Once the student pauses and the

timer expires without further changes, the canvas state is serialized from React state into a structured plan object and sent to the backend via a POST request if no plan exists yet for the student, or a PUT request if a plan already exists. The backend writes or updates the corresponding document in the plans collection and returns a success response, after which the frontend displays a brief autosave confirmation indicator such as "Saved just now" to let the student know their plan is up to date. This approach ensures that the student's work is continuously preserved without flooding the backend with requests during active interactions.

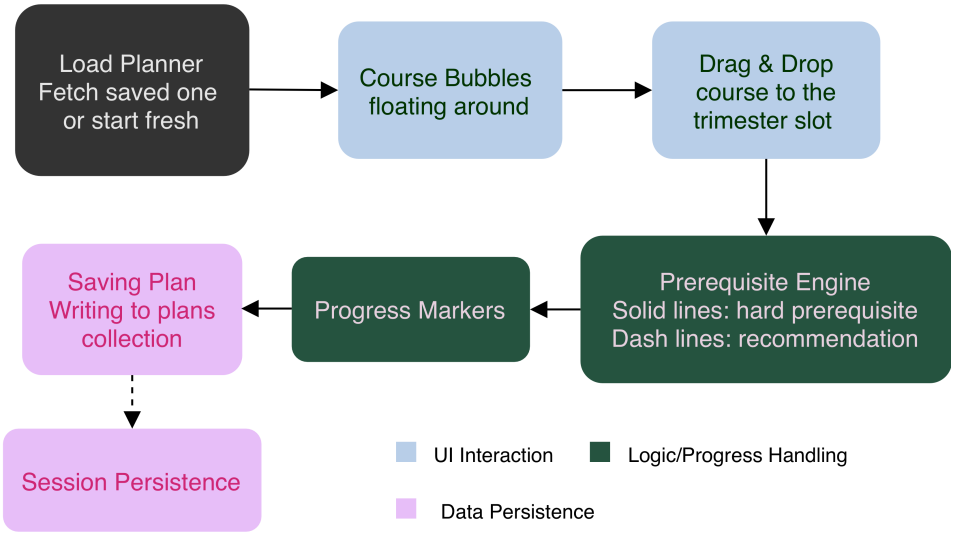

Figure 3.8 Super Planner Pipeline

3.4.3 Progress Markers

The courses that are not yet placed in the plan will also display a marker to suggest the users of their choice of course enrollment, for example, a second year course that has never been enrolled by the third year student will show a marker and a suggestion to the student that they should consider taking it as soon as possible.

3.4.4 Frontend

The Super Planner interface is built as an interactive drag-and-drop canvas within React. The canvas is organized as a grid of semester slots, each representing one trimester, with multiple semesters laid out horizontally across the screen. Course

icons for Major Required courses, Engineering Core courses, and Major Elective courses are presented as a palette of floating icons around the canvas, which students can drag and drop into any semester slot they choose.

General Education courses, I-Design courses, and major elective courses are handled differently from the fixed required courses. Rather than individual course icons, these categories are represented as credit-based placeholder blocks within the plan. Students specify which courses they intend to take to fulfill the credit requirements of each category by selecting from a list within the placeholder, allowing flexibility without cluttering the canvas with every possible elective option.

When a course icon is placed into a semester slot, the interface automatically renders connecting lines to any prerequisite courses already placed on the canvas. Hard prerequisite connections are shown as solid lines, making the dependency chain immediately visible. A secondary set of dashed lines represents faculty suggestions — courses that are not formally required as prerequisites but are recommended to be completed first for better academic preparation.

The canvas also supports marking courses as already completed, allowing students who are mid-program to accurately represent their current academic standing rather than starting from a blank slate. This allows the planner to serve both incoming students mapping out their full degree and continuing students tracking their remaining coursework.

3.4.5 Backend, API, and Database

The backend exposes API endpoints for retrieving a saved plan by student ID, creating a new plan, and updating an existing plan. An additional endpoint serves the full course list with prerequisite and suggestion metadata, which the frontend uses to initialize the course palette and render connecting lines on the canvas.

The plans collection in MongoDB stores one document per student. Each document contains the student's ID, a timestamp of the last update, and a semesters array where each element represents one trimester. Each semester object holds a semester label and a courses array containing the course codes placed in that slot along with a boolean flag indicating whether the course has been marked as completed.

CHAPTER 4
RESULT

This chapter reports your experiment results or study report proposed in previous chapter. It typically consists of 1) Results in terms of tables or figures and 2) explanation or discussion of your results, study, or key findings.

4.1 Results

(Experiment/Results in the forms ofrtables or/and figures)

4.2 Discussions

(Interpretation or the meaning of your experiment/results)

CHAPTER 5
CONCLUSION

This chapter summarizes all of your work. It typically starts with the brief explanation of of what you do such as the objective of your work, design, data, experiment results, key findings, interpretation of your experiment and/or result. The obstacles of your work can also be discussed and finally followed by future work.

5.1 Conclusion

...

5.2 Obstacles

...

5.3 Future Work

...

REFERENCES

[1] Rate my professors. https://www.ratemyprofessors.com/. Accessed: 2024-05-20.

[2] P. R. Aldrich. The curriculum prerequisite network: a tool for visualizing and analyzing academic curricula. *arXiv preprint arXiv:1408.5340*, pages 1--10, 08 2014. doi: 10.48550/arXiv.1408.5340. Subjects: Social and Information Networks (cs.SI); Computers and Society (cs.CY).

[3] M. R. de Oliveira, P. J. C. de Mendonca, A. F. G. Leardine, V. P. Gonçalves, B. F. Medina, and C. G. da Silva. Courseviewer -a prototype for visualizing undergraduate courses and student transcripts. In *Software Engineering and Information Systems Laboratory, School of Technology, University of Campinas*, Limeira, Brazil, 2018. Email: celmar@ft.unicamp.br.

[4] E. Fergan and T. M. Tashu. Course review sentiment analysis: A comparative study of machine learning and deep learning methods. pages 1--7, 2023. doi: 10.1109/BESC59560.2023.10386515.

[5] S. Inc. Stellic: Degree management and student success platform. https://www.stellic.com/, n.d. Accessed: 2025-02-11.

[6] E. C. L.P. Ellucian degree works. https://www.ellucian.com/solutions/degree-works, n.d. Accessed: 2025-02-11.

[7] M. S. Medina, W. T. Smith, S. Kolluru, E. A. Sheaffer, and M. DiVall. A review of strategies for designing, administering, and using student ratings of instruction. *American Journal of Pharmaceutical Education*, 83(5):7177, 06 2019. doi: 10.5688/ajpe7177. PMCID: PMC6630867; PMID: 31333266.

[8] T. Ngubane and J.-R. Tapamo. Tableextractnet: A model of automatic detection and recognition of table structures from unstructured documents. *Informatics*, 11(4):77, 2024. doi: 10.3390/informatics11040077.

[9] M. L. Pandini, Z. Arifin, and D. M. Khairina. Design web service academic information system based multiplatform. In *2014 The 1st International Conference on Information Technology, Computer, and Electrical Engineering (ICITACEE)*, pages 297--302, Semarang, Indonesia, 11 2014. doi: 10.1109/ICITACEE.2014.7065760.

[10] M. Trippel and R. Röpke. Developing a graph-based visualization of elective courses to support course selection in higher education, 2025. URL https://dl.gi.de/handle/20.500.12116/47141.

[11] T. Vaidya. Design and implementation of a digital notice board and forum discussion system. *Global International Multidisciplinary Research Journal (GIMRJ)*, XII(I(VIII)):P097, 06 2024. ISSN 2394-8426. doi: 10.69758/GIMRJ2406I8V12P097. Special Issue On Advancements and Innovations in Computer Application.

[12] M. K. Warsi, K. Baghel, C. Gupta, and S. Alphonse. Course selling website using mern stack. In *2025 International Conference on Innovative Trends in Information Technology (ICITIIT)*, pages 1--5, Kottayam, India, 02 2025. doi: 10.1109/ICITIIT64777.2025.11040918.

[13] Z. Zhu, Y. Zhou, and Q. Luo. A hyper-heuristic algorithm based on genetic and greedy strategy for university course scheduling problem. *Applied Soft Computing*, pages 1--26, 2025. doi: 10.1016/j.asoc.2025.114150.